

ChingiRingi — Backend Guide (Plain English)

Who is this for? You (the client). No tech jargon. Just what the backend does, how it's organised, and what's working right now.

Last updated: 2026-04-21 **Version:** 1.1

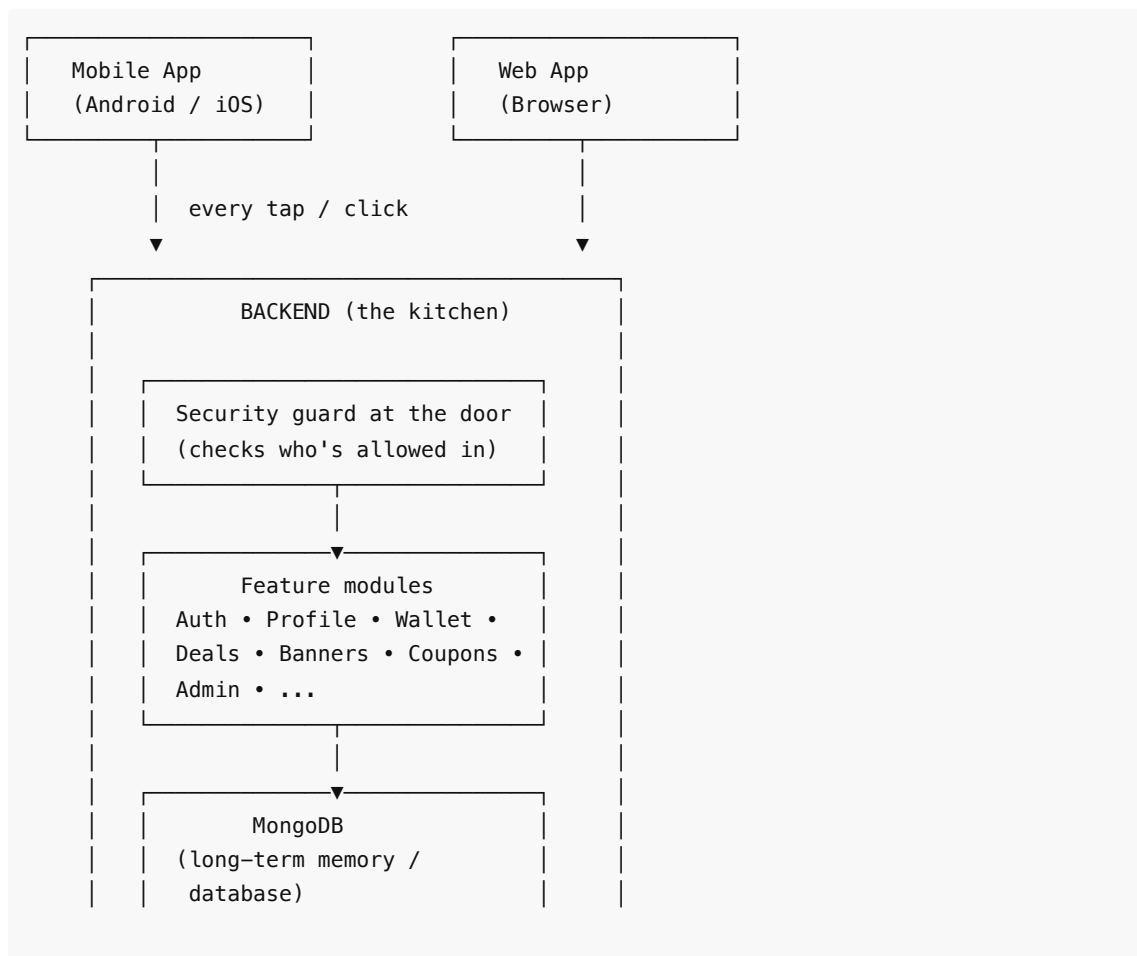
1. What is the backend?

Think of the backend as the **restaurant kitchen**. The mobile app and web app are the waiters who take your order. The kitchen is where the actual cooking happens — storing your account, remembering your wallet balance, checking if a coupon is valid, sending OTPs, and so on.

Every time a user taps a button in the app, a message quietly travels to our kitchen, something gets cooked, and the answer comes back.

- **Where it lives:** hosted on Render (cloud service).
- **What it speaks:** a language called REST / JSON (standard across the industry).
- **What it remembers:** everything goes into a MongoDB database.
- **Security:** HTTPS, rate-limiting, password encryption, and signed cookies.

2. How the pieces fit together



Every request goes through **three layers**:

1. **Security guard** — is this a real user? Are they hitting us too fast?
2. **Module** — the feature they're asking about (e.g. wallet).
3. **Database** — the long-term memory where everything is stored.

3. The security layer (always on)

Guard	What it does	Why it matters
Helmet	Adds standard safety headers	Stops common web attacks
CORS	Only allows our own apps to talk to the backend	Other websites can't use our kitchen
Rate limit	Max 100 requests per 15 min per user (5/min for login)	Stops bots and brute-force attempts
JWT tokens	Gives each user a signed digital wristband	Proves who you are on every visit
Secure cookies (web)	Wristband is kept in a locked cookie	Can't be stolen by other websites
SecureStore (mobile)	Wristband is kept in phone's encrypted vault	Survives app restarts, safe from other apps
Password hashing	Passwords are scrambled before storing	Even we can't read your password

4. What each module does

4.1 Auth (sign up, log in, OTP, password reset)

Status:  Live

This is the **front door**. Covers every way a user can enter the app.

What it handles

- **Sign up** — new user creates an account with name + phone/email + password.
- **Log in** — existing user returns with username + password.
- **OTP login** — user gets a 6-digit code on their phone (for passwordless login on mobile).
- **Forgot password** — resets via email/OTP.
- **Refresh token** — silently keeps user logged in for days/weeks without re-entering password.
- **Logout** — clears the wristband (token) and ends the session.
- **Who am I?** — app asks on startup to confirm the user is still logged in.

Extra protections

- Login throttled to 5 tries per minute → stops password guessers.

- Tokens expire automatically so stolen tokens become useless quickly.
-

4.2 User Profile

Status:  Live

Each user has a profile with their name, phone, email, referral code, and role.

What it handles

- View profile.
 - Edit profile (name, email, etc.).
 - Delete account (permanent).
-

4.3 Addresses

Status:  Live

Users can save multiple delivery addresses (like Amazon / Flipkart).

What it handles

- List all saved addresses.
 - Add a new address.
 - Edit an address.
 - Delete an address.
 - Mark one as the default.
-

4.4 Deals (the product catalog)

Status:  Live

This is the **shop shelf**. Everything users browse and click.

What it handles

- Browse all deals (with filters/pagination).
 - See "Featured" deals (hand-picked).
 - See "Trending brands".
 - Open one deal for full details.
 - Track clicks (for commission analytics).
 - **Admin only:** create / edit / delete deals.
-

4.5 Categories

Status:  Live

Folders that group deals together (Electronics, Fashion, Food, etc.).

What it handles

- List all categories (for the shop navbar).
 - Open one category to see its deals.
 - **Admin only:** create / edit / delete categories.
-

4.6 Banners

Status:  Live

The big promotional images on the homepage (carousel / hero banners).

What it handles

- Fetch active banners to display.
 - **Admin only:** upload, edit, delete, or toggle active/inactive.
-

4.7 Wallet & Transactions

Status:  Live

Each user has a wallet that holds their earned cashback / rewards / referral bonuses.

What it handles

- See current wallet balance.
- See a summary (total earned, total withdrawn, pending).
- See the full list of transactions (credits and debits).
- Open one transaction for details.

Every credit/debit is stored as a separate **transaction record** — nothing is ever overwritten, so we always have a full audit trail.

4.8 OTP (one-time passwords)

Status:  Live (used by Auth)

Not a user-facing module — it's the **behind-the-scenes** plumbing that generates, stores, and verifies the 6-digit codes used for:

- Login via phone
- Password reset
- Phone verification

Codes expire after a few minutes and can only be used once.

4.9 Coupons

Status:  Live (backend) — mobile form for new fields being added

This is where discount codes like `WELCOME50` or `CASHBACK100` live.

What it handles today

- **Admin** creates a coupon with:
 - Code (e.g. `WELCOME50`)
 - Type — percentage or flat rupees
 - Value + max discount cap
 - Minimum order value
 - Start + expiry date
 - Total usage limit + per-user usage limit
- **Admin** can list, search, edit, delete, or toggle active.

- **Admin** sees a usage dashboard for each coupon:
 - Total redemptions, unique users, total discount given
 - Revenue generated from coupon orders, average order value
 - Last 30 days timeline (daily bar chart data)
 - Top 10 users by redemption count
- **Customer** can:
 - **Validate** a code at checkout (dry-run: "will this work? how much off?") — no usage consumed.
 - **Apply** a code (actually redeems it). Backend checks validity, increments usage counter atomically (race-safe), and logs who used what, when, on which order.

Safety nets

- Percentage discounts are capped at 100%.
- Discounts never make the final amount negative.
- If two customers try to use the last available slot at the exact same moment, only one gets it — the other sees a clear error.
- Every redemption is permanently logged — nothing is ever overwritten, so we always have a full audit trail.

Right now: backend is live. The mobile form is being updated to collect the new "per-user limit" and "start date" fields, and the admin usage dashboard screen is being built next.

4.10 Admin panel

Status:  Live (expanding)

Locked-down area only admins can enter. Used for running the business.

What it handles today

- Dashboard stats (counts of users, deals, etc.)
- List all users (with search + pagination)
- List all deals

Coming in next iterations

- Coupons CRUD + usage dashboard
 - Withdrawal approvals (Payouts)
 - Orders
 - Inventory
 - Full user management (block/unblock, role changes)
-

5. How the app remembers things (the database)

We use **MongoDB** — a flexible database that stores information as "documents" (like index cards).

Each module has its own drawer of index cards:

- `users` — one card per user
- `deals` — one card per deal
- `categories` — one card per category
- `banners` — one card per banner
- `wallets` — one card per user's wallet

- `transactions` — one card per credit/debit
- `addresses` — one card per saved address
- `otps` — short-lived cards for 6-digit codes
- `coupons` — one card per coupon
- `couponRedemptions` — one card every time a coupon is used

We never delete important history. Even if a deal is removed, the transactions tied to it stay — so we always have an audit trail.

6. A typical user journey (end-to-end)

Let's follow **Priya** using the app.

1. **Opens app** → app asks backend "who is Priya?" → backend checks her wristband → returns her profile. (**Auth + Profile**)
2. **Sees banners + featured deals on home** → backend returns active banners and top deals. (**Banners + Deals**)
3. **Taps a deal → taps "Get Offer"** → backend records the click for commission tracking. (**Deals**)
4. **Goes back, opens wallet** → backend returns her balance + last 10 transactions. (**Wallet**)
5. **Adds a new address for delivery** → backend saves it. (**Addresses**)
6. **At checkout, enters coupon `FESTIVE25`** → backend checks: is it active? not expired? she hasn't used it too many times? discount calculated → applied. (**Coupons — coming soon**)
7. **Next day opens app** → wristband still valid, straight into home. (**Auth — refresh token**)

7. What's live vs what's next

Feature	Status
Sign up / Log in / OTP	✓ Live
Password reset	✓ Live
User profile	✓ Live
Addresses	✓ Live
Deals browsing	✓ Live
Deal click tracking	✓ Live
Categories	✓ Live
Banners	✓ Live
Wallet + transactions	✓ Live
Admin: dashboard + users + deals	✓ Live
Admin: banners / categories / deals CRUD	✓ Live
Coupons: admin CRUD (backend)	✓ Live
Coupons: validate + apply at checkout	✓ Live

Coupons: usage analytics endpoint	✅ Live
Coupons: mobile form (new fields) + admin usage screen	🔧 Polishing
Withdrawals / Payouts	🔧 Next
Orders + Inventory	🔧 Planned
Notifications (push / email)	➡️ SOON Planned
Referrals	➡️ SOON Planned

8. How we keep this document fresh

- This doc is updated **at every checkpoint** (after a feature lands or changes shape).
- Before updating, we will **ask you first** so you always know what's about to change.
- The PDF version lives alongside this file and is regenerated on each update.

9. Changelog

Date	Version	What changed
2026-04-21	1.0	Initial document covering all live modules + coupons-in-progress
2026-04-21	1.1	Coupons backend shipped: admin CRUD, validate + apply at checkout (race-safe), usage analytics (totals, 30-day timeline, top 10 users). Mobile form + admin usage screen next.

ChingiRingi Backend — built on Node.js + Express + MongoDB, hosted on Render.